



Retrieval Basics: Building a Simple Search System

Module 04: Retrieval before Generation

GenAI, NLP and Prompt Engineering

Quick Recap: Why We Need Retrieval

- LLMs generate answers by predicting the next most likely word in a sequence.
- This capability is powerful but has limitations.
- LLMs do not store your specific documents like a traditional database.
- They rely on patterns learned during training.
- These patterns can be outdated or incomplete.
- Without direct data access, models may guess, speculate, or hallucinate information.
- This information can be plausible but factually incorrect.
- Retrieval systems solve this by supplying verified, relevant text passages.
- This grounds responses in real data before the model generates an answer.



Module Overview and Learning Outcomes

01

Understand Retrieval as the Foundation of RAG

Learn how retrieval forms the critical first step in Retrieval-Augmented Generation systems, bridging the gap between stored knowledge and AI generation.

03

Learn Embedding and Similarity Workflows

Understand how text converts to numerical vectors and how similarity ranking identifies the most relevant information for any query.

02

Master Chunking Strategies

Discover why breaking documents into optimal-sized chunks matters for search accuracy, context preservation, and answer quality.

04

Retrieve Top Chunks for Grounded Answers

Apply the complete retrieval pipeline to fetch verified passages that enable accurate, hallucination-free AI responses.

Why Normal LLMs Are Not Enough

The Hallucination Problem

LLMs may return answers that sound authoritative and confident but are completely incorrect. They can invent outdated details, fabricate statistics, or create imaginary information that has no basis in reality.

For example, when asked about library operating hours, an LLM might confidently state "9 AM to 5 PM" when the actual hours are completely different. The model generates what seems most probable based on training patterns, not factual truth.

Retrieval systems eliminate guessing by providing the LLM with verified, official data from your actual documents before generating any response.



Without Retrieval

Question: "What are the library hours on Saturday?"

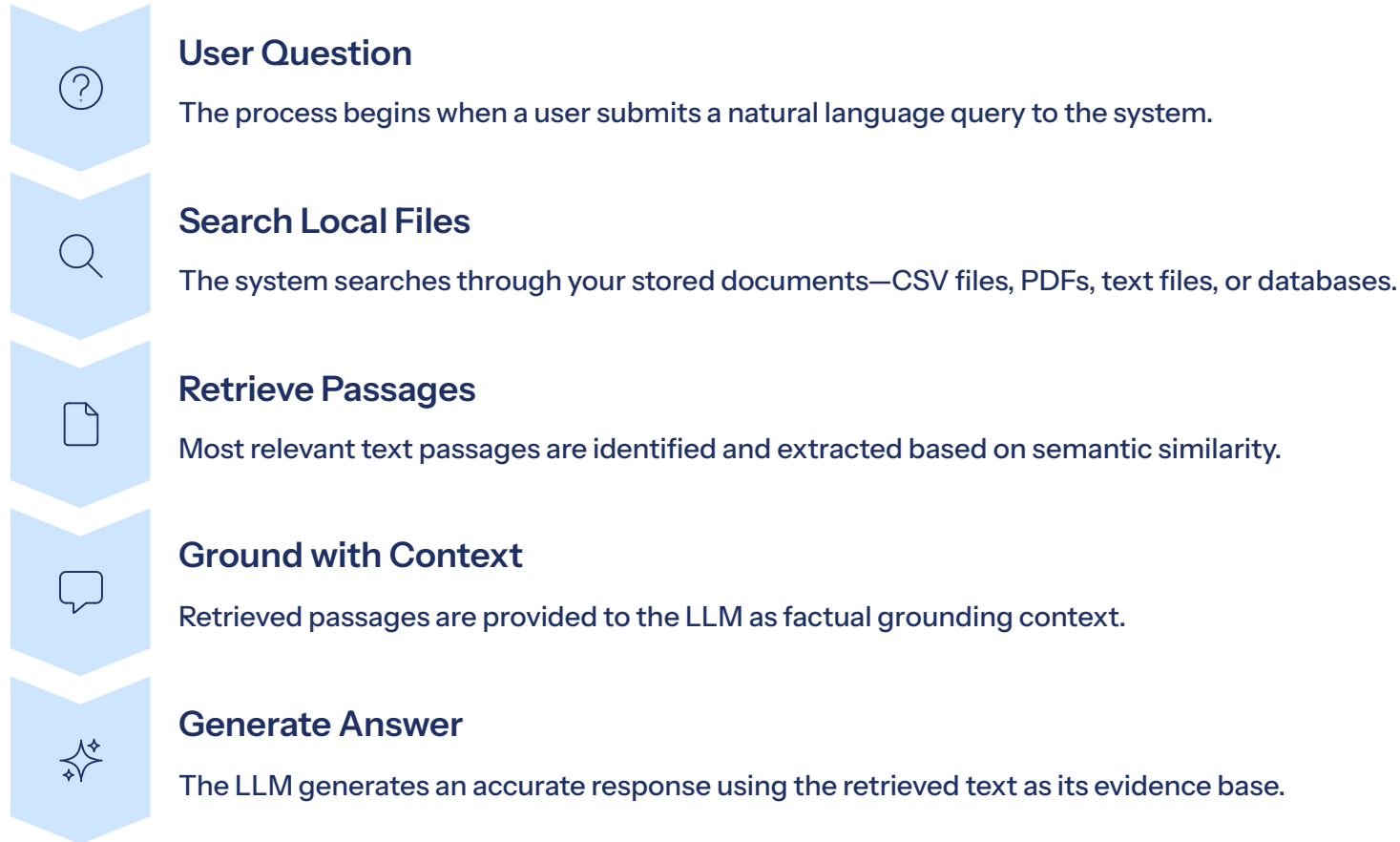
LLM Guess: "The library is open from 9:00 AM to 5:00 PM on Saturdays."

With Retrieval

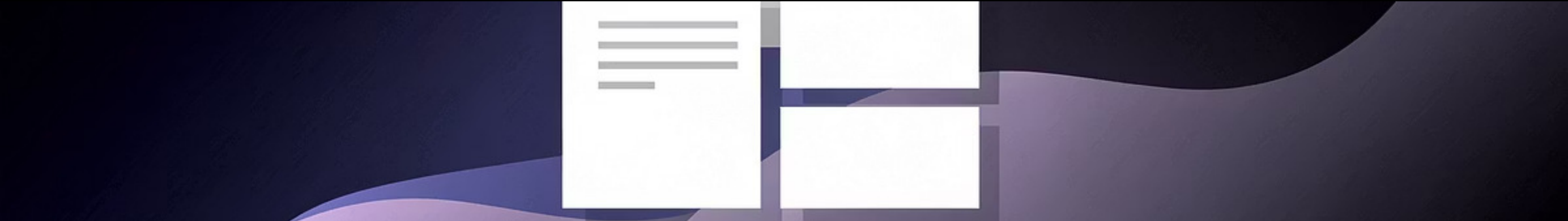
Retrieved Data: "Saturday hours: 10:00 AM - 2:00 PM"

Grounded Answer: "According to our records, the library is open from 10:00 AM to 2:00 PM on Saturdays."

Retrieval Before Generation: System Flow



This pipeline ensures every generated answer is traceable back to specific source documents, dramatically reducing hallucinations and improving trustworthiness. The retrieval step acts as a fact-checking mechanism that happens automatically before any content generation.



Chunking Basics: Breaking Documents into Searchable Pieces

What is Chunking?

Chunking is the process of splitting large documents into smaller, manageable passages or segments. This fundamental preprocessing step transforms unwieldy documents into searchable units.

Why chunk? Searching through entire documents is slow and imprecise. Smaller chunks enable faster, more targeted matching and clearer context for the LLM.

1

Complete Ideas

Each chunk should contain a coherent, self-contained concept or paragraph.

2

Faster Search

Smaller units allow the system to quickly scan and compare semantic meaning.

3

Better Matching

Good chunks improve the quality and relevance of search results significantly.

Chunk Size and Overlap: Quality Controls

Too Large

- Chunks mix multiple unrelated topics together
- Matching becomes less precise
- Accuracy decreases as context becomes diluted
- Harder to identify the exact relevant passage

Optimal Size

- **Recommended: 100–200 words**
- **Or 500–800 tokens**
- Balances context with precision
- Contains complete thoughts
- Enables accurate semantic matching

Too Small

- Chunks lose important surrounding context
- Incomplete information leads to confusion
- Answers may lack necessary details
- Increased risk of fragmented results

The Overlap Strategy

Use **15–30% overlap** between consecutive chunks to prevent information loss at boundaries. When a key sentence falls exactly at a chunk boundary, overlap ensures it appears in at least one chunk completely.

Example with 50-word overlap:

Chunk 1: Words 1–150

Chunk 2: Words 120–270

Chunk 3: Words 240–390

Words 120–150 appear in both Chunk 1 and Chunk 2, ensuring continuity.

Embedding the Chunks: Storing Meaning as Numbers

Embeddings are the secret weapon of modern semantic search. Each text chunk is converted into a numerical vector—typically an array of 768 or 1536 dimensions—that captures its semantic meaning.

The magic: Chunks with similar meanings produce vectors that are mathematically close together in high-dimensional space, even if they use completely different words. This enables meaning-based search rather than simple keyword matching.

These embedding vectors are stored in arrays, CSV files, or NumPy matrices for efficient retrieval. When a query arrives, the system can rapidly compare the query vector against all stored chunk vectors to find semantic matches.



Example: Stored Chunks with Embeddings

ID	Chunk Text	Embedding Vector (simplified)
1	The library opens at 10 AM on weekdays and closes at 6 PM.	[0.23, 0.87, -0.12, 0.45, ...]
2	Weekend hours are different: Saturday 10 AM to 2 PM, Sunday closed.	[0.19, 0.91, -0.08, 0.41, ...]

Query Embedding and Similarity Matching

1

Convert Query to Embedding

The user's question is transformed into an embedding vector using the same embedding model used for chunks, ensuring mathematical compatibility.

2

Compare with All Chunks

The query vector is compared against every stored chunk vector in the database. This comparison happens in milliseconds even with thousands of chunks.

3

Compute Similarity Scores

A similarity metric (typically cosine similarity) calculates how close each chunk vector is to the query vector, producing a score for each chunk.

4

Select Top-K Results

The system ranks all chunks by similarity score and selects the top 3-5 most relevant chunks for context.

5

Ground the LLM Answer

The selected chunks are passed to the LLM as verified context, enabling it to generate an accurate, evidence-based response.

Cosine Similarity and Ranking

Understanding the Metric

Cosine similarity measures the angle between two vectors in high-dimensional space. It produces a score between 0 and 1, where higher scores indicate closer semantic meaning.

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

This metric is ideal for text because it focuses on direction (meaning) rather than magnitude (length), making it robust across different text lengths.

Score Interpretation

Score Range	Match Quality	Interpretation
0.80 - 1.00	Strong Match	Highly relevant, semantically very similar
0.60 - 0.79	Moderate Match	Related content, potentially useful
0.40 - 0.59	Weak Match	Tangentially related, may lack relevance
0.00 - 0.39	Poor Match	Unrelated or irrelevant content

The Ranking Process

After computing similarity scores for all chunks, the system sorts them in descending order. The ranking identifies the best matching chunk (highest score) and the top-k chunks for comprehensive context. This ranked list ensures the LLM receives the most relevant information first.

Python Demo: NumPy Retrieval Logic

Here's the core retrieval logic that powers RAG systems. This simplified example demonstrates how to compute cosine similarity and identify the best matching chunk using NumPy arrays.

```
import numpy as np

# Query and chunk vectors (simplified to 5 dimensions)
query_vector = np.array([0.5, 0.8, 0.1, 0.3, 0.9])
chunk_vectors = np.array([
    [0.4, 0.7, 0.2, 0.4, 0.8], # Chunk 1
    [0.6, 0.9, 0.0, 0.2, 0.95], # Chunk 2 (best match)
    [0.1, 0.3, 0.8, 0.7, 0.2] # Chunk 3
])

# Compute cosine similarity
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Find best chunk
scores = [cosine_similarity(query_vector, chunk) for chunk in chunk_vectors]
best_idx = np.argmax(scores)
print(f"Best chunk: {best_idx}, Score: {scores[best_idx]:.3f}")
```

Key Concepts

- Query and chunks represented as NumPy arrays
- Cosine similarity function computes dot product divided by vector magnitudes
- List comprehension calculates scores for all chunks efficiently

Real-World Extensions

- Scale to thousands of chunks using vectorized operations
- Implement top-k retrieval with `np.argsort()`
- Add threshold filtering to exclude low-scoring chunks

This is the fundamental retrieval mechanism used in production RAG systems. While real implementations use specialized vector databases and optimized libraries, the underlying mathematics remains the same.

Retrieval Tuning and Failure Modes

Optimization Strategies

Top-K Retrieval

Retrieve 3–5 chunks instead of just one. Multiple chunks provide richer context and catch information spread across documents. Too few chunks risk missing context; too many introduce noise.

Threshold Filtering

Remove chunks with similarity scores below 0.60 (or your chosen threshold). This prevents weakly related or irrelevant content from polluting the LLM's context window.

Common Failure Modes

Over-Retrieval

Problem: Too many chunks (10+) overwhelm the LLM with information.

Result: Model confusion, contradictory information, degraded answer quality.

Solution: Limit top-k to 3-5 most relevant chunks.

Under-Retrieval

Problem: Only 1 chunk retrieved, or threshold too strict.

Result: Missing critical information, incomplete answers, increased hallucination risk.

Solution: Retrieve multiple chunks, lower threshold carefully.

Quality Issues

Problem: Irrelevant chunks, duplicate content, or contradictory passages retrieved.

Result: Confused or incorrect LLM outputs.

Solution: Improve chunking strategy, deduplicate, validate embedding quality.

The key is finding the sweet spot: retrieve enough context for completeness without overwhelming the model with noise. Regular testing and tuning against real queries helps optimize these parameters for your specific use case.

Debrief and Homework

Debrief Discussion

Why Retrieval First?

Explain why retrieval must happen before LLM answering. What problems does it solve?

Chunking Benefits

Discuss what chunking improves in search quality and why chunk size matters.

Similarity Scoring

How does cosine similarity determine relevance? What makes a "good" score?

Retrieval Risks

Identify one major risk if retrieval goes wrong. How would it affect answer quality?

Homework Assignment

Hands-On Exercise:

1. Create or obtain a small FAQ CSV file with 5–10 question-answer pairs
2. Treat each FAQ entry as a chunk (or split longer ones into chunks)
3. Write 3 test queries related to your FAQ content
4. For each query, document your results in this format:
QUERY → TOP CHUNK → SIMILARITY SCORE
5. Read ahead: RAG pipeline overview (next module)

This exercise builds practical intuition for retrieval quality and helps you understand how query-chunk matching works in real scenarios.

Thank You

Ready to Build RAG Systems

You now understand the foundational retrieval concepts that power modern AI applications. From chunking strategies to cosine similarity, you've gained practical knowledge of how to ground LLM responses in verified data.

Next Steps: Apply these concepts in the homework exercise, then explore full RAG pipeline implementation in the upcoming modules.

MODULE 04 COMPLETE

→ NEXT: RAG PIPELINE

